

Лабораторная работа № 14

Взваров Михаил, НПИбд-01-25

Лабораторная работа № 14

Студент: Взваров Михаил

Группа: НПИбд-01-25

Цель работы

Получить практические навыки написания командных файлов в оболочке Bash, работы с системной справочной информацией, генерации случайных последовательностей и организации синхронизации процессов с помощью файлового семафора.

Выполнение работы

1. Командный файл `myman.sh`

Был создан командный файл `myman.sh`, предназначенный для поиска и отображения справочной страницы указанной команды непосредственно из каталога `/usr/share/man/man1`.

Скрипт принимает имя команды как аргумент командной строки, проверяет наличие каталога со справочными страницами и ищет соответствующий файл с возможными расширениями `.gz`, `.bz2` и `.xz`.

Исходный код программы представлен на рисунке.

После запуска скрипта с именем команды `ls` была найдена и открыта соответствующая справочная страница.

2. Генерация случайной последовательности символов

Был создан командный файл `random_letters.sh`, генерирующий последовательность случайных букв.

Для генерации используются встроенная переменная Bash `RANDOM`, строки с буквами латинского алфавита и функции для получения случайной строчной или заглавной буквы. Длина последовательности передаётся первым аргументом, а второй аргумент определяет использование заглавных букв.

```

5 if [ $# -ne 1 ]; then
6     echo "Использование: $0 <команда>"
7     echo "Пример: $0 ls"
8     exit 1
9 fi
10
11 CMD="$1"
12
13 if [ ! -d "$MAN_DIR" ]; then
14     echo "Ошибка: каталог $MAN_DIR не найден"
15     exit 1
16 fi
17
18 MAN_FILE=""
19 for ext in "" ".gz" ".bz2" ".xz"; do
20     if [ -f "$MAN_DIR/${CMD}${ext}" ]; then
21         MAN_FILE="$MAN_DIR/${CMD}${ext}"
22         break
23     fi
24 done
25
26 if [ -z "$MAN_FILE" ]; then
27     echo "Справка для команды '$CMD' не найдена"
28     echo "Проверьте: man $CMD"
29     exit 1
30 fi
31
32 echo "=== Справка для команды '$CMD' ==="
33 echo "Источник: $MAN_FILE"
34 echo "-----"
35
36 if command -v less &> /dev/null; then
37     less "$MAN_FILE"
38 else
39     case "$MAN_FILE" in
40         *.gz) gunzip -c "$MAN_FILE" ;;
41         *.bz2) bunzip2 -c "$MAN_FILE" ;;
42         *.xz) unxz -c "$MAN_FILE" ;;
43         *) cat "$MAN_FILE" ;;
44     esac
45 fi
46
47 michailvzvarov@fedora:~/reports/lab1$

```

Рис. 1: Листинг myman.sh

```

==> (lesspipe 2.22) append : to filename to view the original troff file
ls(1)
User Commands
NAME
ls - list directory contents
SYNOPSIS
ls [OPTION]... [FILE]...
DESCRIPTION
List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.
Mandatory arguments to long options are mandatory for short options too.
-a, --all
do not ignore entries starting with .
-A, --almost-all
do not list implied . and ..
--author
with -l, print the author of each file
-b, --escape
print C-style escapes for nongraphic characters
--block-size=SIZE
with -l, scale sizes by SIZE when printing them; e.g., '--block-size=M'; see SIZE format below
-B, --ignore-backups
do not list implied entries ending with ~
-c with -lt: sort by, and show, ctime (time of last change of file status information); with -l: show ctime and sort by name; otherwise: sort by ctime, newest
-C list entries by columns
--color[=WHEN]
color the output WHEN; more info below
-d, --directory
list directories themselves, not their contents
/usr/share/man/man1/ls.1.gz

```

Рис. 2: Результат выполнения myman.sh

Исходный код программы представлен на рисунке.

```
17 USE_UPPER=${2:-0}
18
19 echo "Генерация случайной последовательности длиной $LENGTH..."
20
21 if [ "$USE_UPPER" -eq 1 ]; then
22     for i in $(seq 1 $LENGTH); do
23         random_letter_upper
24     done
25 else
26     for i in $(seq 1 $LENGTH); do
27         random_letter
28     done
29 fi
30
31 echo ""
32
33 echo ""
34 echo "=== Разные варианты ==="
35
36 echo -n "Строчные: "
37 for i in $(seq 1 15); do
38     random_letter
39 done
40 echo ""
41
42 echo -n "Заглавные: "
43 for i in $(seq 1 15); do
44     random_letter_upper
45 done
46 echo ""
47
48 echo -n "Смешанные: "
49 for i in $(seq 1 20); do
50     if [ $((RANDOM % 2)) -eq 0 ]; then
51         random_letter
52     else
53         random_letter_upper
54     fi
55 done
56 echo ""
57
nichilvzvarov@fedora:~/reports/lab14$
```

Рис. 3: Листинг random_letters.sh

В результате выполнения скрипта была получена случайная последовательность заданной длины. Также программа выводит примеры последовательностей из строчных, заглавных и смешанных букв.

3. Синхронизация процессов с помощью семафора

Был создан командный файл `semaphore.sh`, демонстрирующий синхронизацию нескольких процессов при доступе к общему ресурсу.

В качестве семафора используется lock-файл `/tmp/semaphore.lock`. Для атомарного создания файла применяется режим `noclobber`. Если файл успешно создан, процесс получает доступ к ресурсу, использует его заданное время и затем удаляет lock-файл. Если файл уже существует, другой процесс ожидает и повторяет попытку.

Исходный код программы представлен на рисунке.

Для проверки одновременно были запущены два экземпляра скрипта. Один процесс получил ресурс, а второй обнаружил занятость ресурса и перешёл в состояние ожидания. После освобождения семафора возможность захвата ресурса получила другая программа.

Вывод

В ходе лабораторной работы были закреплены навыки программирования в оболочке Bash. Был реализован собственный способ обращения к системным справочным страницам, изучена генерация случайных последователь-

```
Генерация случайной последовательности длиной 25...  
KFXESGTLpQFWQEBEXOLGWJSK
```

```
*** Разные варианты ***  
Строчные: hdxkexdxfngx  
Заглавные: TNDCFUNEQFPPOSJ  
Смешанные: eNYJLFKYJgQSjOpDITva  
michailvzvarov@fedora:~/reports/lab14$
```

Рис. 4: Результат выполнения random_letters.sh

```
1 #!/bin/bash  
2  
3 WAIT_TIME=${1:-5}  
4 USE_TIME=${2:-3}  
5 PROCESS_ID=${3:-1}  
6 LOCK_FILE="/tmp/semaphore.lock"  
7  
8 log() {  
9     echo "[$(date +%H:%M:%S)] [Процесс $PROCESS_ID] $1"  
10 }  
11  
12 log "Запущен. Время ожидания: $WAIT_TIME сек, время использования: $USE_TIME сек"  
13  
14 while true; do  
15     log "Пытаюсь захватить ресурс..."  
16  
17     if ( set -o noclobber; echo "$$" > "$LOCK_FILE" ) 2>/dev/null; then  
18         # Ресурс захвачен  
19         log "Ресурс захвачен! Использую в течение $USE_TIME секунд..."  
20         sleep $USE_TIME  
21         rm -f "$LOCK_FILE"  
22         log "Ресурс освобожден"  
23     else  
24         log "Ресурс занят, ожидаю $WAIT_TIME секунд..."  
25         sleep $WAIT_TIME  
26     fi  
27 done  
28  
michailvzvarov@fedora:~/reports/lab14$
```

Рис. 5: Листинг semaphore.sh

```
1 #!/bin/bash
2
3 WAIT_TIME=${1:-5}
4 USE_TIME=${2:-3}
5 PROCESS_ID=${3:-1}
6 LOCK_FILE="/tmp/semaphore.lock"
7
8 log() {
9     echo "[$(date +%H:%M:%S)] [Процесс $PROCESS_ID] $1"
10 }
11
12 log "Запущен. Время ожидания: $WAIT_TIME сек, время использования: $USE_TIME сек"
13
14 while true; do
15     log "Пытаюсь захватить ресурс..."
16
17     if ( set -o noclobber; echo "$$" > "$LOCK_FILE" ) 2>/dev/null; then
18         # Ресурс захвачен
19         log "Ресурс захвачен! Использую в течение $USE_TIME секунд..."
20         sleep $USE_TIME
21
22         rm -f "$LOCK_FILE"
23         log "Ресурс освобожден"
24     else
25         log "Ресурс занят, ожидаю $WAIT_TIME секунд..."
26         sleep $WAIT_TIME
27     fi
28 done
29
mikhailvzvarov@fedora:~/reports/lab14$ rm -f /tmp/semaphore.lock
mikhailvzvarov@fedora:~/reports/lab14$ cd ~/reports/lab14
chmod +x semaphore.sh
./semaphore.sh 2 5 1
[23:00:17] [Процесс 1] Запущен. Время ожидания: 2 сек, время использования: 5 сек
[23:00:17] [Процесс 1] Пытаюсь захватить ресурс...
[23:00:17] [Процесс 1] Ресурс захвачен! Использую в течение 5 секунд...
[23:00:22] [Процесс 1] Ресурс освобожден
[23:00:22] [Процесс 1] Пытаюсь захватить ресурс...
[23:00:22] [Процесс 1] Ресурс захвачен! Использую в течение 5 секунд...
```

Рис. 6: Работа двух процессов с семафором

ностей с использованием переменной `RANDOM`, а также рассмотрен механизм синхронизации процессов на основе файлового семафора.

Практическая работа показала возможности Bash для автоматизации системных задач и организации взаимодействия нескольких процессов.

Ответы на контрольные вопросы

- 1. Что представляет собой командный файл?**
Командный файл — это текстовый файл, содержащий последовательность команд оболочки, которые выполняются интерпретатором команд.
- 2. Для чего используется переменная `RANDOM` в Bash?**
Переменная `RANDOM` возвращает псевдослучайное целое число и может применяться для генерации случайных значений и последовательностей.
- 3. Для чего используются функции в Bash?**
Функции позволяют объединять повторяющиеся действия в именованный блок команд, который можно вызывать многократно.
- 4. Что такое семафор?**
Семафор — механизм синхронизации, используемый для управления доступом нескольких процессов к общему ресурсу.
- 5. Для чего нужен lock-файл?**
Lock-файл служит признаком того, что ресурс в данный момент занят

одним из процессов.

6. **Как работает цикл `while`?**

Цикл `while` выполняет набор команд до тех пор, пока проверяемое условие возвращает истинный код завершения.

7. **Что делает команда `sleep`?**

Команда `sleep` приостанавливает выполнение процесса на указанное количество времени.